Team Project & Task Management System

Backend code changes — Part B and Part C

Copy-ready additions for the supplied Spring Boot WebFlux + R2DBC backend. This document continues the earlier guide: Part A (signup request and admin approval) is assumed to be completed already. Part B adds a secure single-request task feed; Part C covers member-list performance and safe verification.

Important: Make a Git commit/backup first. The code below is based on the supplied project layout and current authorization model. Add the snippets to the named files; do not replace entire classes unless the section explicitly says so. Compile each service after its changes. The internal project-ID endpoint must only be reachable service-to-service, never directly exposed publicly.

Part B — Faster task loading

B1. Add project repository query

File: **project-service/src/main/java/com/hartford/tms/project/repo/ProjectRepo.java**. Add the method below to the existing interface; keep the current imports and methods.

```
import reactor.core.publisher.Flux;

// Add inside ProjectRepo
Flux<Project> findByManagerId(Long managerId);
```

If Flux is not already imported, add the import shown above. This query is used only by the internal visibility lookup.

B2. Add service method to resolve authorized project IDs

File: **project-service/src/main/java/com/hartford/tms/project/service/ProjectService.java**. Add this method inside ProjectService. It mirrors the existing access policy: ADMIN sees all projects; PROJECT_MANAGER sees projects they manage; TEAM_MEMBER sees projects they belong to.

```
public reactor.core.publisher.Mono<List<Long>> visibleProjectIds(Long userId, String role) {
    if (userId == null || role == null) {
        return reactor.core.publisher.Mono.error(
                new com.hartford.tms.project.exceptions.BadRequestException(
                        "userId and role are required"));
    }

    if ("ADMIN".equals(role)) {
        return projectRepo.findAll()
                .map(Project::getId)
                .collectList();
    }

    if ("PROJECT_MANAGER".equals(role)) {
        return projectRepo.findByManagerId(userId)
                .map(Project::getId)
                .collectList();
    }

    if ("TEAM_MEMBER".equals(role)) {
        return memberRepo.findByUserId(userId)
                .map(ProjectMember::getProjectId)
                .distinct()
                .collectList();
    }

    return reactor.core.publisher.Mono.error(
            new com.hartford.tms.project.exceptions.ForbiddenException(
                    "Unsupported role for project visibility"));
}
```

The class already imports List, Project, ProjectMember and Mono. If the IDE reports a missing import, use the fully qualified Mono above or add reactor.core.publisher.Mono. The returned list is the authorization boundary; do not return all project IDs to a non-admin.

B3. Add an internal handler

File: **project-service/src/main/java/com/hartford/tms/project/handler/ProjectHandler.java**. Add this method inside ProjectHandler. The Task Service will call it; the gateway must not route /internal/**.

```
public Mono<ServerResponse> visibleProjectIds(ServerRequest request) {
    final Long userId;
    final String role = request.queryParam("role").orElse("");
    try {
        userId = Long.valueOf(request.queryParam("userId")
                .orElseThrow(() -> new BadRequestException("userId is required")));
```

```
    } catch (NumberFormatException ex) {
        return Mono.error(new BadRequestException("userId must be a number"));
    }
    return projectService.visibleProjectIds(userId, role).flatMap(this::ok);
}
```

This route is internal, so the task-service supplies these values from RequestUser derived from gateway-validated headers. Network policy should prevent untrusted clients from calling project-service directly.

B4. Register the internal route

File: **project-service/src/main/java/com/hartford/tms/project/router/ProjectRouter.java**. Add the route next to the existing internal project route:

```
.GET("/internal/projects/visible", projectHandler::visibleProjectIds)
```

Call format: GET http://project-service/internal/projects/visible?userId=12&role=PROJECT_MANAGER

B5. Add Task Service client call

File: **task-service/src/main/java/com/hartford/tms/task/client/ProjectClient.java**. Add these imports and method. Keep the existing getProject(...) method.

```
import java.util.List;

public Mono<List<Long>> getVisibleProjectIds(Long userId, String role) {
    return webClient.get()
            .uri(uriBuilder -> uriBuilder
                    .path("/internal/projects/visible")
                    .queryParam("userId", userId)
                    .queryParam("role", role)
                    .build())
            .retrieve()
            .bodyToFlux(Long.class)
            .collectList();
}
```

If your WebClient response decoder does not accept bodyToFlux for JSON arrays, this is the intended contract: the project service returns a JSON array such as [1,2,3]. Do not send userId/role from query parameters received by the public task request; pass only actor.id() and actor.role() from RequestUser.

B6. Add task repository methods

File: **task-service/src/main/java/com/hartford/tms/task/repo/TaskRepo.java**. Add imports and methods. Keep existing repository methods.

```
import org.springframework.data.domain.Pageable;
import java.util.Collection;

// Add inside TaskRepo
Flux<Task> findByProjectIdIn(Collection<Long> projectIds, Pageable pageable);
Mono<Long> countByProjectIdIn(Collection<Long> projectIds);
```

Spring Data R2DBC derives the IN query from the method name. If your Spring Data version does not support Pageable on a derived reactive query, implement an explicit @Query with LIMIT/OFFSET for your database dialect and add a matching count query; do not fall back to an unfiltered findAll().

B7. Add service method for the visible task page

File: **task-service/src/main/java/com/hartford/tms/task/service/TaskService.java**. Add the method inside TaskService. Required imports: java.util.Collections (if you choose to use it) is not needed by the snippet below.

```
public Mono<PageResponse<TaskResponse>> visibleTasks(RequestUser actor, Pageable pageable) {
    return projectClient.getVisibleProjectIds(actor.id(), actor.role())
            .flatMap(projectIds -> {
                if (projectIds == null || projectIds.isEmpty()) {
                    return Mono.just(PageResponse.of(List.of(), pageable, 0L));
                }
                return Mono.zip(
                        taskRepo.findByProjectIdIn(projectIds, pageable)
                                .map(TaskResponse::from)
                                .collectList(),
                        taskRepo.countByProjectIdIn(projectIds))
                        .map(result -> PageResponse.of(
                                result.getT1(), pageable, result.getT2()));
            });
}
```

This first version intentionally returns all tasks within authorized projects. If your member policy should show only assigned tasks rather than every task in a project, change the authorization/business rule explicitly and add the corresponding repository filter. Do not silently change existing project screen behavior.

B8. Add handler method and route

File: **task-service/src/main/java/com/hartford/tms/task/handler/TaskHandler.java**. Add this method:

```
public Mono<ServerResponse> visibleTasks(ServerRequest request) {
    return RequestUser.require(request)
            .flatMap(user -> taskService.visibleTasks(user,
                    Params.page(request, SORTABLE, "updatedAt,desc", Sort.Direction.DESC)))
            .flatMap(this::ok);
}
```

File: **task-service/src/main/java/com/hartford/tms/task/router/TaskRouter.java**. Add this fixed route before .GET("/tasks/{id}", ...):

```
.GET("/tasks/visible", taskHandler::visibleTasks)
```

The frontend can now call GET /tasks/visible?page=0&size=50. Keep size between 1 and 100. Because /tasks/visible is a normal authenticated task route, do not add it to public gateway paths.

Part C — Member list and loading delay

The supplied User Service already has paginated search/count operations for GET /admin/users and GET /users/directory. Do not add a second member endpoint or use blocking calls. First make sure the frontend is requesting page 0 with a size no larger than 100 and is reading the PageResponse.content array.

C1. Keep admin user search paginated

File: **user-service/src/main/java/com/hartford/tms/user/handler/UserHandler.java**. The existing listUsers method is already correct in principle. If yours differs, replace the method with this version; it preserves the existing role and active filters and caps page size through Params.page.

```
public Mono<ServerResponse> listUsers(ServerRequest request) {
    return RequestUser.require(request, "ADMIN")
            .flatMap(admin -> userService.search(
                    Params.optional(request, "q"),
                    Params.optional(request, "role"),
                    activeParam(request),
                    Params.page(request, SORTABLE, "id,asc", Sort.Direction.ASC)))
            .flatMap(this::ok);
}
```

Do not replace this with userRepo.findAll().collectList(): that loads every user into memory and removes pagination.

C2. Keep search and count reactive

File: **user-service/src/main/java/com/hartford/tms/user/service/UserService.java**. The current search method already fetches a page and its count reactively. Keep this implementation; if it was changed, restore it:

```
public Mono<PageResponse<UserResponse>> search(
        String q, String role, Boolean active, Pageable pageable) {
    return Mono.zip(
            userSearchRepo.search(q, role, active, pageable)
                    .map(UserResponse::from)
                    .collectList(),
            userSearchRepo.count(q, role, active))
        .map(result -> PageResponse.of(
                result.getT1(), pageable, result.getT2()));
}
```

File: **user-service/src/main/java/com/hartford/tms/user/repo/UserSearchRepo.java**. Keep its search SQL and count SQL filters identical. The count query must apply the same q/role/active conditions as the row query; otherwise pagination totals are wrong. Do not add a second count call in the handler.

C3. Confirm the existing page size limit

File: **user-service/src/main/java/com/hartford/tms/user/util/Params.java**. Keep the existing validation rule page ≥ 0 and size 1–100. Apply the same rule in task-service Params. The screenshot error is caused by size=200; the frontend should send size=100 or lower.

C4. Add safe timing logs to identify slow requests

If requests still remain pending, add lightweight duration logs around the actual service operation. Do not log passwords, JWTs, or personal details.

```
// Example in UserHandler.listUsers: add imports
import java.time.Duration;
import java.time.Instant;

// Replace return chain temporarily with a timed version:
public Mono<ServerResponse> listUsers(ServerRequest request) {
    Instant started = Instant.now();
    return RequestUser.require(request, "ADMIN")
            .flatMap(admin -> userService.search(
                    Params.optional(request, "q"),
                    Params.optional(request, "role"),
                    activeParam(request),
                    Params.page(request, SORTABLE, "id,asc", Sort.Direction.ASC)))
            .doFinally(signal -> org.slf4j.LoggerFactory
                    .getLogger(UserHandler.class)
                    .info("GET /admin/users completed signal={} durationMs={}",
                            signal, Duration.between(started, Instant.now()).toMillis()))
            .flatMap(this::ok);
}
```

After diagnosing, you may keep or remove the timing log. If the service log reports fast completion but the browser spinner remains, inspect the frontend subscription/loading-finally logic. If it reports slow completion, inspect SQL query plans, database connectivity and the User Service logs.

Part C — database/index and runtime checks

C5. Inspect indexes before adding new ones

The provided schema already has indexes for user role/active and task project/status, assignee and due date. Avoid adding duplicate indexes blindly. In the database, inspect the actual schema and use EXPLAIN for the SQL in UserSearchRepo and TaskSearchRepo. Add an index only when the query plan and table size justify it.

```
-- MySQL examples; run against your development database first
SHOW INDEX FROM users;
SHOW INDEX FROM tasks;
EXPLAIN SELECT id, name, email, role, active
FROM users WHERE active = 1 ORDER BY name LIMIT 100;
EXPLAIN SELECT id, project_id, title, status, updated_at
FROM tasks WHERE project_id = 1 ORDER BY updated_at DESC LIMIT 50;
```

C6. Build and test in this order

1. Add ProjectRepo, ProjectService, ProjectHandler and ProjectRouter changes; build project-service.

2. Add ProjectClient, TaskRepo, TaskService, TaskHandler and TaskRouter changes; build task-service.

3. Test GET /tasks/visible?page=0&size;=50 with an ADMIN token, a PROJECT_MANAGER token, and a TEAM_MEMBER token. Confirm each sees only authorized projects.

4. Test size=101 returns HTTP 400; size=100 succeeds. Do not use size=200.

5. Test GET /admin/users?page=0&size;=20 and GET /users/directory?page=0&size;=20. Check that the JSON has content, page, size, totalElements, totalPages and last.

6. In browser DevTools → Network, compare request duration with the service timing logs. A frontend spinner is not fixed by increasing backend timeouts.

7. Ensure /internal/projects/visible is not routed by API Gateway and project-service is not publicly exposed. If service ports are reachable from user devices, secure this endpoint with service authentication or network policy before deployment.

C7. Maven commands

```
# From each service directory, use the project's wrapper if present
./mvnw test
./mvnw package

# Or, if Maven is installed and there is no wrapper
mvn test
mvn package
```

Important compatibility note: The code follows the package names and reactive patterns in your supplied source. Spring Data R2DBC pageable derived queries can vary by version/dialect. If `findByProjectIdIn(Collection<Long>, Pageable)` fails to compile or paginate correctly, implement that two-query operation using the database dialect in your project and test it; never replace it with an unrestricted task query.

End of Part B and Part C code guide. Part A changes are not repeated here.